A Midterm Progress Report

on

Blockchain Secure Vault

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

Computer Science Engineering

SUBMITTED BY

EKAMPREET SINGH 2302515 | GAURAV PRASHAR 2302518 | GAURAV SHARMA 2302519

UNDER THE GUIDANCE OF

ER. MEETALI

(MARCH-2026)



Department of Computer Science and Engineering

GURU NANAK DEV ENGINEERING COLLEGE, LUDHIANA

Contents

1 Introduction **3**
- 1.1 Project Overview . . . 3
- 1.2 Technology Used . . . 3
- 1.3 Field of Project . . . 3
- 1.4 Special Technical Terms . . . 3
- 1.5 Objectives of the Project . . . 4

2 System Requirements **5**
- 2.1 Hardware Requirements . . . 5
- 2.2 Software Requirements . . . 5
- 2.3 Browser Requirements . . . 5

3 Software Requirement Analysis **6**
- 3.1 Problem Definition . . . 6
- 3.2 Identity & Access Module . . . 6
- 3.3 Secure Document Processing Module . . . 6
- 3.4 Blockchain Integrity Module . . . 6
- 3.5 Sharing & Retrieval Module . . . 6
- 3.6 Key Non-Functional Requirements . . . 7

4 Software Design **8**
- 4.1 Design Approach Overview . . . 8
- 4.2 Objective 1: Smart Contract for SHA-256 Hash Storage (Immutability & Transparency) . . . 8
- 4.3 Objective 2: AES-256 Encryption and IPFS Decentralized Storage . . . 9
- 4.4 Objective 3: Secure Document Dashboard (View, Verify, Download) . . . 9
- 4.5 Objective 4: Document Validity Verification . . . 10

5 Testing Module **12**
- 5.1 Proposed Testing Techniques . . . 12
  - 5.1.1 Unit Testing . . . 12
  - 5.1.2 Integration Testing . . . 12
  - 5.1.3 Smart Contract Testing . . . 12
  - 5.1.4 API Testing . . . 13
  - 5.1.5 Security Testing . . . 13
  - 5.1.6 End-to-End (E2E) Testing . . . 13
  - 5.1.7 Performance and Load Testing . . . 13
  - 5.1.8 Usability and Compatibility Testing . . . 14
- 5.2 System Test Cases . . . 14

6 Performance of the Project (Developed So Far) **16**
- 6.1 Conclusion . . . 16

7 Output Screen **17**

List of Figures

1 Control flow of Blockchain Secure Vault 11
2 Landing page of the system before user authentication 17
3 User registration interface for creating a new account 17
4 Account registration using Google authentication 18
5 User login interface using username and password 18
6 User login using Google authentication 19
7 User login using One-Time Password (OTP) verification 19
8 Home interface displayed after user authentication 20
9 Secure document management dashboard after successful login 20
10 Document upload interface for secure file encryption and storage 21
11 Document verification interface for validating file authenticity using blockchain records 21

1 Introduction

1.1 Project Overview

This project presents a secure and user-friendly platform for document storage with verifiable integrity. Conventional cloud storage solutions often prioritize convenience over security, leading to risks related to confidentiality and undetectable tampering [1]. To overcome these limitations, the proposed system integrates end-to-end encryption with blockchain-based verification, ensuring that documents remain private while their authenticity can be independently validated.

Users can securely register, upload encrypted documents, and verify file integrity by comparing cryptographic hashes with immutable on-chain records. A modern web interface, combined with a robust backend, manages encryption, access control, and secure sharing workflows. The system aims to make strong security practices and trustworthy document verification accessible to individuals and organizations without requiring specialized expertise.

1.2 Technology Used

- **Frontend:** Next.js with TypeScript
- **Backend:** Node.js and Express for REST APIs, authentication, and file handling
- **Blockchain:** Solidity smart contracts using Hardhat for on-chain hash storage and verification
- **Cryptography:** AES-256 for encryption and SHA-256 for hashing [2]
- **File Processing:** Mammoth.js for DOCX files, PDF.js for PDF rendering, and Web Crypto API for client-side decryption

1.3 Field of Project

- Information Security and Web Application Development
- Blockchain-based Integrity Verification
- Secure and Compliance-Oriented Document Management

1.4 Special Technical Terms

- **End-to-End Encryption (E2EE):** Files are encrypted before upload and decrypted only by authorized users.
- **Content Hash:** A cryptographic fingerprint used to detect file modifications.
- **On-chain Verification:** Validation of file integrity using immutable blockchain records.
- **Smart Contract:** Blockchain program that stores and exposes document hashes.
- **Immutable Ledger:** Tamper-resistant blockchain record ensuring data integrity.

1.5 Objectives of the Project

The primary objectives of the proposed system are as follows:

1. To design and deploy smart contracts on a blockchain platform for storing SHA-256 document hashes, ensuring immutability and transparency of records.
2. To encrypt documents using the AES-256 encryption standard and store the encrypted files on IPFS to provide secure and decentralized storage.
3. To develop a secure document management dashboard that allows users to upload, view, verify, and download documents through a browser-based interface.
4. To implement a verification mechanism that allows users to confirm the authenticity and integrity of documents by comparing their cryptographic hashes with blockchain records.

2 System Requirements

This section describes the hardware and software requirements necessary to develop and run the Blockchain Secure Document Vault system.

2.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 (8th Generation) / AMD Ryzen 3 or higher
RAM	Minimum 4 GB (Recommended 8 GB)
Storage	Minimum 2 GB free disk space (Recommended SSD with at least 10 GB free space)
Network	Stable internet connection for blockchain interaction, IPFS storage, and email OTP services

2.2 Software Requirements

Software	Requirement
Operating System	Windows 10/11, Linux, or macOS
Code Editor	Visual Studio Code
Runtime Environment	Node.js (v18 or higher)
Backend Framework	Express.js
Frontend Framework	Next.js (React with TypeScript)
Database	MongoDB
Blockchain Development	Hardhat and Ethers.js
API Testing Tool	Postman
Version Control	Git and GitHub

2.3 Browser Requirements

Browser	Requirement
Google Chrome	Latest Version
Microsoft Edge	Latest Version
JavaScript	Must be enabled

3 Software Requirement Analysis

3.1 Problem Definition

Organizations and individuals need a way to store sensitive documents so they are confidential, tamper-evident, and easy to verify later. Traditional cloud storage can protect availability, but it does not natively provide immutable proof of document authenticity. The required system is a secure document vault that encrypts files, stores encrypted payloads off-chain, and records document fingerprints on blockchain for trustable verification.

3.2 Identity & Access Module

- User registration and login (password, Google, OTP).
- Email verification with OTP before full access.
- JWT-based session management, protected APIs, logout/token invalidation.
- Profile retrieval and access control for user-owned resources.

3.3 Secure Document Processing Module

- File intake with validation (type/size checks).
- Hash generation (SHA-256) for integrity identity.
- AES-256 encryption before external storage.
- Encrypted upload to decentralized/object storage and metadata persistence.
- Key-handling rule: decryption key shown once to user, not stored server-side.

3.4 Blockchain Integrity Module

- Smart contract stores mapping of file hash $\rightarrow$ storage reference.
- On upload: record hash transaction and persist transaction reference.
- On verify: recompute hash from uploaded file and compare with on-chain record.
- Return authenticity status (verified / not found / mismatch) with audit-ready metadata.

3.5 Sharing & Retrieval Module

- Owner can share document access with registered recipients.
- Recipient can view/download encrypted content with out-of-band decryption key.
- Owner can revoke active share access.
- Document dashboard for “my documents,” “shared with me,” and “shared by me.”

3.6 Key Non-Functional Requirements

- **Security:** Encryption at rest, strict authentication checks, and minimal secret exposure.
- **Reliability:** Graceful handling of blockchain, IPFS, or service failures.
- **Performance:** Responsive upload and verification flows for typical file sizes.
- **Usability:** Simple workflows for upload, key copying, verification, sharing, and revocation.

4 Software Design

This section presents the detailed design of the Blockchain Secure Document Vault system. The design addresses all four objectives defined in the project synopsis and explains how each objective is technically achieved.

4.1 Design Approach Overview

The system follows a hybrid architecture combining:

- Blockchain (for immutability and transparency)
- IPFS (for decentralized encrypted file storage)
- AES-256 encryption (for confidentiality)
- Web-based dashboard (for secure interaction)

The architecture ensures that:

- Documents remain confidential
- Hash records remain immutable
- Verification remains transparent
- Users maintain ownership and access control

4.2 Objective 1: Smart Contract for SHA-256 Hash Storage (Immutability & Transparency)

To ensure tamper-proof document verification, a smart contract named `DocumentStorage` is deployed on the blockchain.

Design Details:
When a user uploads a document:

- The system generates a SHA-256 hash of the original file.
- The smart contract stores the following metadata:
  - File Hash (bytes32)
  - IPFS CID
  - Owner Address
  - Timestamp

Core Functions:

- `storeDocumentHash(hash, ipfsCID)`
  Stores document metadata and rejects duplicate hashes.
- `verifyDocumentHash(hash)`
  Returns whether the document exists and its associated metadata.

Design Outcome:

- Immutability is ensured because blockchain records cannot be altered.
- Transparency is achieved because transaction hashes are publicly auditable.

4.3 Objective 2: AES-256 Encryption and IPFS Decentralized Storage

To maintain confidentiality and secure storage, the system encrypts documents before uploading them to IPFS.

Design Steps:

1. File is received by backend.
2. SHA-256 hash is generated.
3. File is encrypted using:
    - AES-256 algorithm
    - Unique randomly generated key
    - Random Initialization Vector (IV)
4. Only encrypted file is uploaded to IPFS.
5. Plaintext file is never stored externally.

Security Rule:

- The encryption key is displayed once to the user.
- The server does not store the decryption key.

Design Outcome:

- Confidentiality is preserved.
- Even if IPFS is accessed, the file remains unreadable.
- Blockchain stores only the hash, not the file content.

4.4 Objective 3: Secure Document Dashboard (View, Verify, Download)

A secure web-based dashboard provides document management functionality.

Features:

1. Upload

- Upload file
- Encryption and blockchain recording
- Display transaction reference

2. View

- Fetch encrypted file from IPFS
- Decrypt in browser using user key
- Render preview

3. Download

- Retrieve encrypted file

- Decrypt using provided key
- Download original file

4. Access Control

- JWT-based authentication
- Users can access only their own or shared documents
- Revocation support for shared access

Design Outcome:

- Secure interaction
- Controlled access
- User-friendly interface

4.5 Objective 4: Document Validity Verification

The system provides a blockchain-based verification mechanism.

Verification Process:

1. User uploads a file for verification.
2. System recomputes SHA-256 hash.
3. Smart contract verification function is called.

If the hash exists:

- Document is marked authentic.
- Blockchain transaction hash is shown.
- Timestamp is displayed.

If not found:

- Document is marked invalid.

Design Outcome:

- Tamper-evident validation
- Audit-ready proof
- Trustable verification mechanism

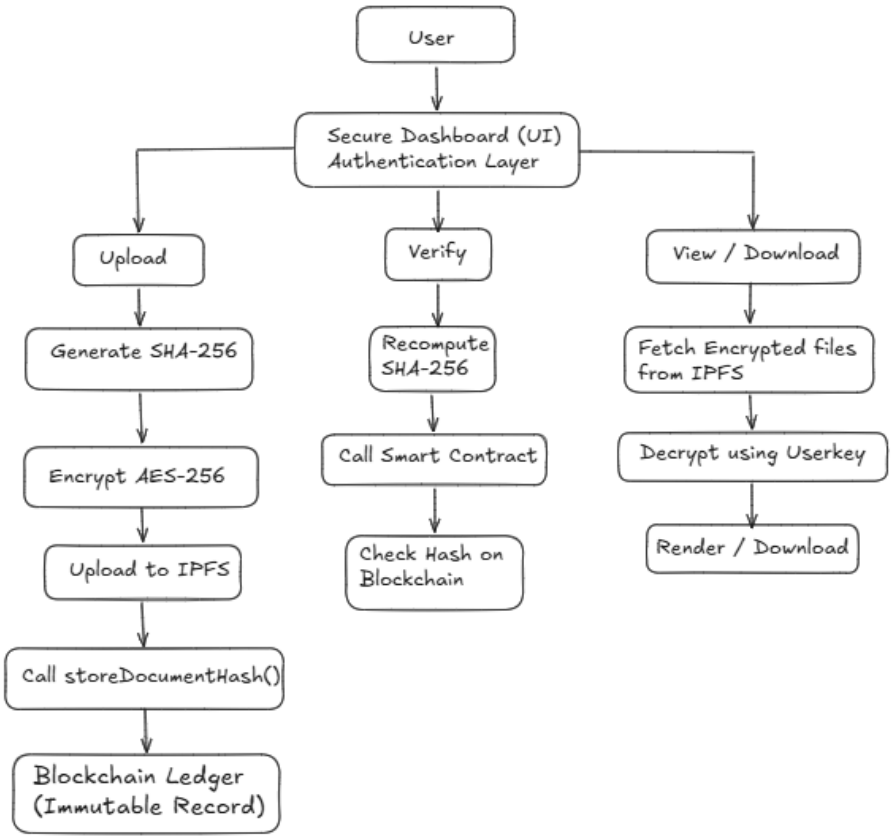


Figure 1: Control flow of Blockchain Secure Vault

5 Testing Module

This section describes the testing techniques proposed for validating the Blockchain Secure Document Vault system. The objective of testing is to ensure correctness, security, reliability, and performance of all system components including smart contracts, backend services, encryption mechanisms, and dashboard functionality.

5.1 Proposed Testing Techniques

To ensure comprehensive system validation, the following testing strategies will be used.

5.1.1 Unit Testing

Unit testing is performed to validate individual components and functions independently.

Modules Covered:

- SHA-256 hashing function
- AES-256 encryption and decryption
- JWT authentication logic
- OTP verification logic
- Smart contract functions (`storeDocumentHash`, `verifyDocumentHash`)

Purpose: To ensure each function performs accurately in isolation.

5.1.2 Integration Testing

Integration testing verifies interaction between connected modules.

Interfaces Tested:

- Backend API $\leftrightarrow$ Database (MongoDB)
- Backend API $\leftrightarrow$ IPFS storage
- Backend API $\leftrightarrow$ Blockchain smart contract
- Dashboard $\leftrightarrow$ Backend API

Purpose: To confirm correct data flow and communication between system components.

5.1.3 Smart Contract Testing

Smart contract testing ensures on-chain correctness.

Scenarios Tested:

- Duplicate hash prevention
- Correct storage of metadata
- Accurate verification response
- Transaction reversion on invalid operations

Purpose: To guarantee immutability and integrity of blockchain records.

5.1.4 API Testing

API endpoints are tested using tools such as Postman.

Validation Includes:

- Correct status codes (200, 401, 403, 409, 500)
- Input validation
- Error handling
- Response format verification

Purpose: To ensure reliable backend services.

5.1.5 Security Testing

Security testing validates access control and protection mechanisms.

Checks Include:

- Unauthorized access denial
- Invalid or expired JWT token rejection
- Role-based access enforcement
- Proper key-handling behavior (no server-side key storage)

Purpose: To protect against unauthorized access and data breaches.

5.1.6 End-to-End (E2E) Testing

Simulates real user workflows.

- Register $\rightarrow$ Verify OTP $\rightarrow$ Login
- Upload $\rightarrow$ Encrypt $\rightarrow$ Store $\rightarrow$ Record on blockchain
- Verify document authenticity
- View and download document
- Share and revoke access

Purpose: To validate complete system functionality under realistic scenarios.

5.1.7 Performance and Load Testing

System performance is evaluated under:

- Multiple concurrent uploads
- Multiple verification requests
- Large file uploads (within allowed limit)

Purpose: To measure system responsiveness and stability.

5.1.8 Usability and Compatibility Testing

Dashboard is tested on:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Mobile and desktop viewports

Purpose: To ensure consistent user experience across platforms.

5.2 System Test Cases

The following test cases will be executed to validate system functionality.

- **TC01 – User Registration (Valid Case)**
  Input: Valid full name, username, email, password
  Expected Result: Account created, OTP sent, user marked as unverified.
- **TC02 – OTP Verification (Valid Case)**
  Input: Correct OTP within expiry period
  Expected Result: Email verified successfully and login permitted.
- **TC03 – Login (Invalid Password)**
  Input: Correct username with incorrect password
  Expected Result: 401 Unauthorized response, no JWT issued.
- **TC04 – Access Protected API Without Token**
  Input: Request to secured endpoint without JWT
  Expected Result: 401 Unauthorized error.
- **TC05 – Upload Valid File**
  Input: Supported file (e.g., PDF within size limit)
  Expected Result:
  - SHA-256 generated
  - AES-256 encryption completed
  - File uploaded to IPFS
  - Hash recorded on blockchain
  - Decryption key displayed once
- **TC06 – Upload Duplicate File**
  Input: Same file uploaded again
  Expected Result: Upload rejected with duplicate error; no second blockchain entry created.
- **TC07 – Upload Unsupported File Type**
  Input: Unsupported extension
  Expected Result: Validation failure with appropriate error message.
- **TC08 – Verify Authentic Document**
  Input: Previously uploaded original file
  Expected Result: Hash matches blockchain record; document marked as verified.

- **TC09 – Verify Tampered or Unknown Document**
  Input: Modified file or never-uploaded file
  Expected Result: Verification fails; document marked invalid or not found.

- **TC10 – Dashboard View (Correct Key)**
  Input: Valid document with correct decryption key
  Expected Result: Successful decryption and document preview.

- **TC11 – Dashboard View (Incorrect Key)**
  Input: Valid document with wrong decryption key
  Expected Result: Decryption fails; file not rendered.

- **TC12 – Download Document (Authorized)**
  Input: Valid key for owned/shared document
  Expected Result: Original decrypted file downloaded successfully.

- **TC13 – Unauthorized Document Access**
  Input: User attempts to access unshared document
  Expected Result: 403 Forbidden or 404 Not Found response.

- **TC14 – Share Document**
  Input: Owner shares document with valid registered user
  Expected Result: Share record created; recipient can access document.

- **TC15 – Revoke Shared Access**
  Input: Owner revokes access
  Expected Result: Recipient can no longer view or download document.

- **TC16 – Smart Contract Immutability Check**
  Input: Attempt to overwrite existing hash
  Expected Result: Transaction reverted; original data unchanged.

- **TC17 – IPFS or Service Failure Handling**
  Input: Simulated IPFS unavailability
  Expected Result: Graceful error response; no inconsistent records created.

- **TC18 – Expired or Blacklisted Token**
  Input: Expired or invalid JWT token
  Expected Result: Request denied; user redirected to login.

6 Performance of the Project (Developed So Far)

At the current stage of development, the system demonstrates stable functional performance across the core modules that have been implemented so far, including user authentication and secure document upload with encryption and storage. These modules are functioning reliably under normal testing conditions and allow users to interact with the system without major delays or interruptions.

The frontend interface is responsive and provides clear feedback during operations. Loading indicators and error messages are displayed during processes such as authentication and file upload. This improves the user experience by keeping users informed about the progress of their actions.

The backend APIs responsible for handling authentication and document upload workflows are functioning correctly. The system successfully processes file uploads, performs SHA-256 hash generation, and encrypts files using the AES-256 encryption standard before storing them in decentralized storage. The integration between the frontend, backend, and storage components is working consistently for the currently implemented features.

The blockchain integration for storing document hash records has also been established. However, blockchain-related operations may take slightly more time compared to traditional database operations because transactions must be confirmed on the network. This delay is expected and acceptable within the scope of a blockchain-based system.

From a security-performance perspective, cryptographic operations such as hash generation and encryption introduce some computational overhead. Despite this, the system performance remains acceptable for the current prototype scale and does not create noticeable delays during testing.

Based on the tests conducted so far, no major performance bottlenecks have been observed in the implemented modules. The system can therefore be considered stable for prototype-level functionality at this stage of development.

6.1 Conclusion

The implemented system demonstrates satisfactory performance for the currently developed modules, particularly in authentication, secure document processing, and encrypted storage. Additional features and further performance evaluation will be conducted in later stages of development.

7 Output Screen



Figure 2: Landing page of the system before user authentication

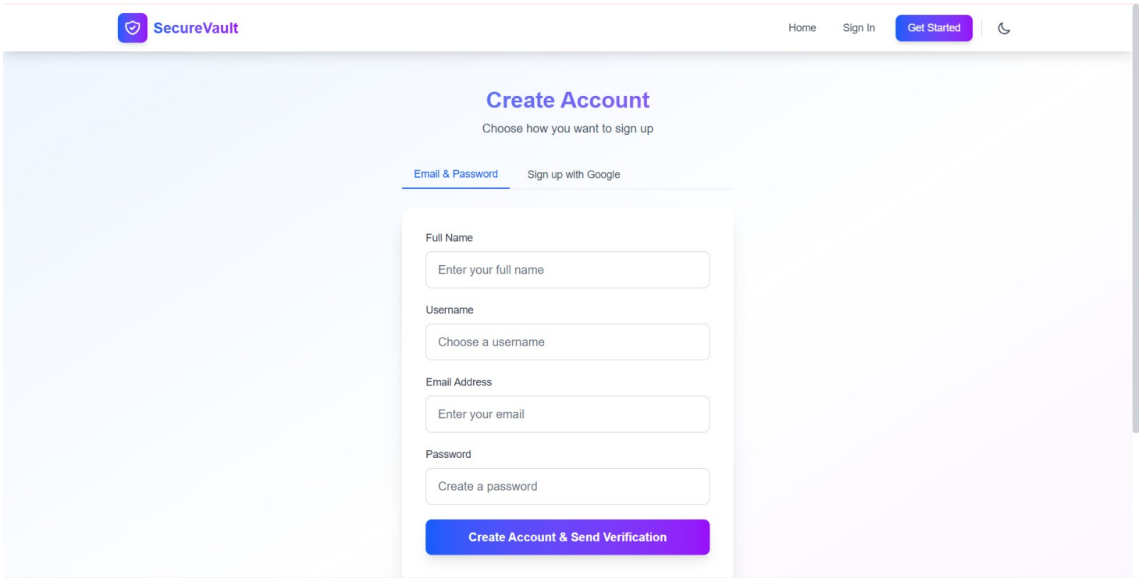


Figure 3: User registration interface for creating a new account

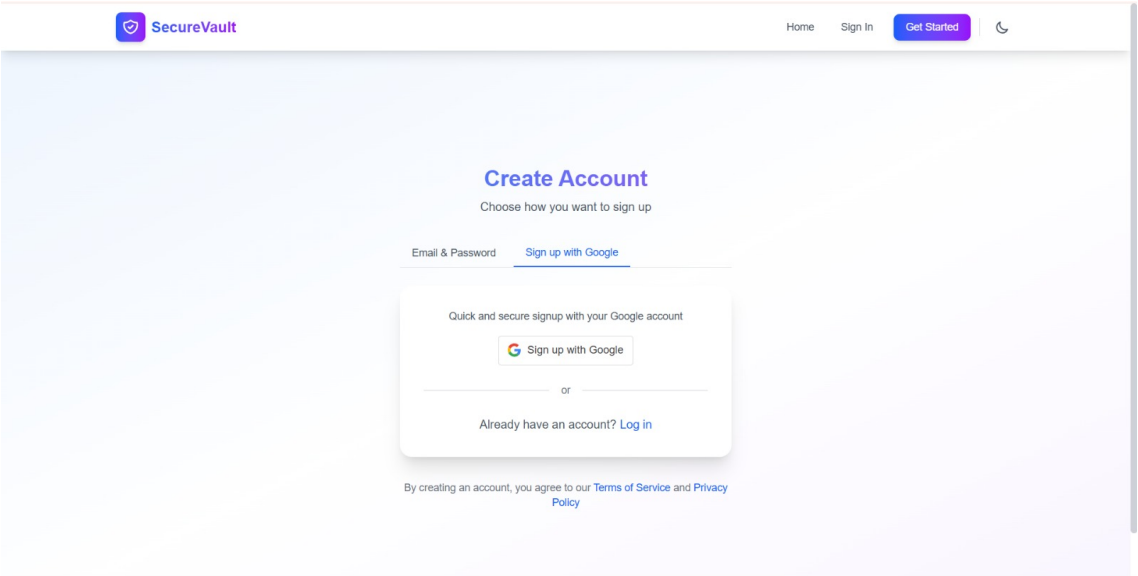


Figure 4: Account registration using Google authentication

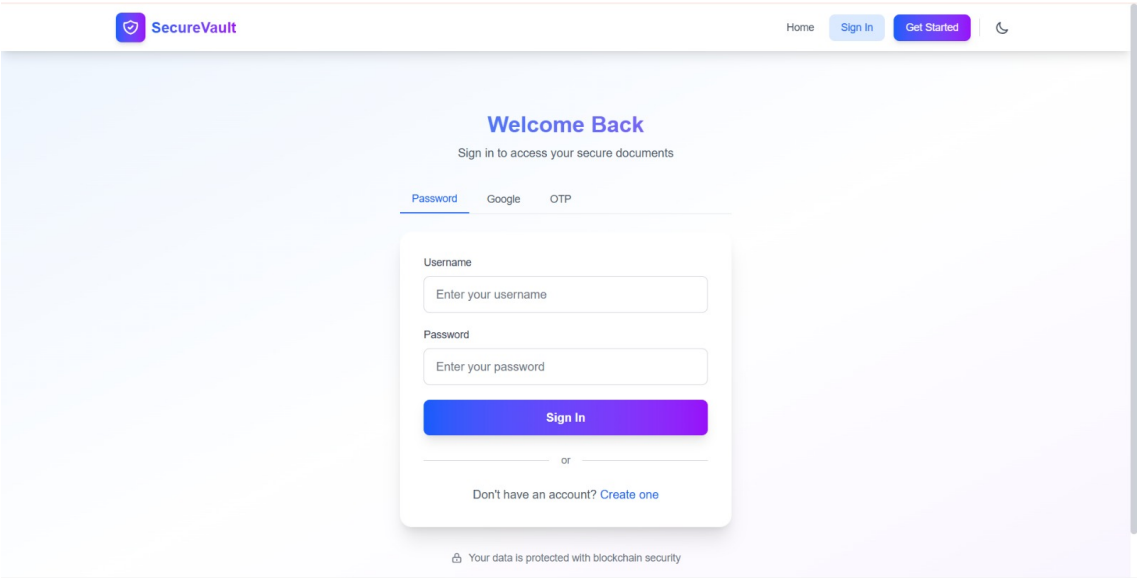


Figure 5: User login interface using username and password

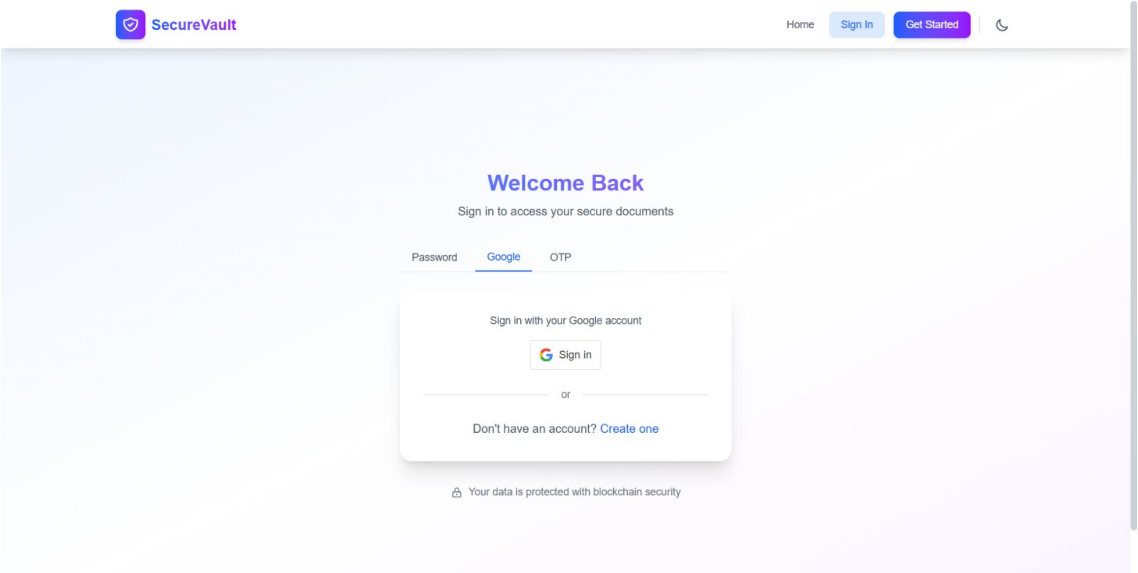


Figure 6: User login using Google authentication

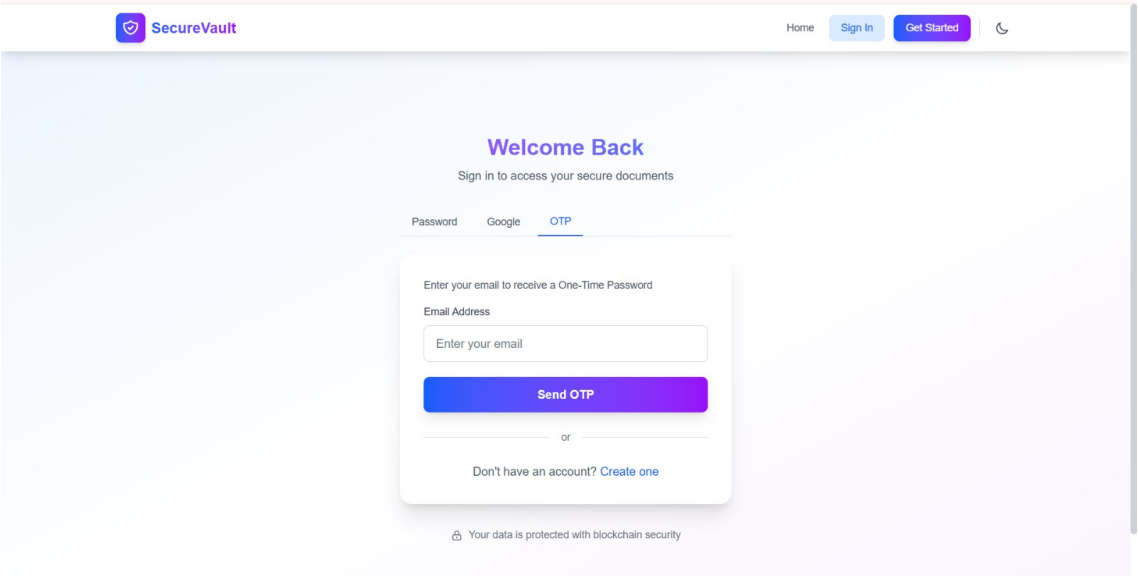


Figure 7: User login using One-Time Password (OTP) verification



Figure 8: Home interface displayed after user authentication

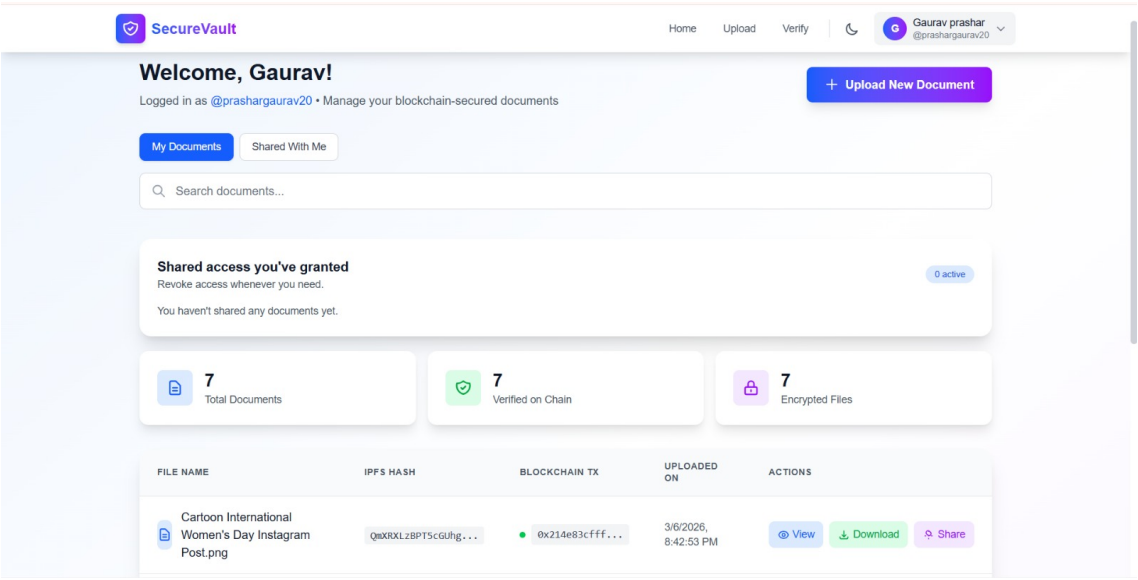


Figure 9: Secure document management dashboard after successful login

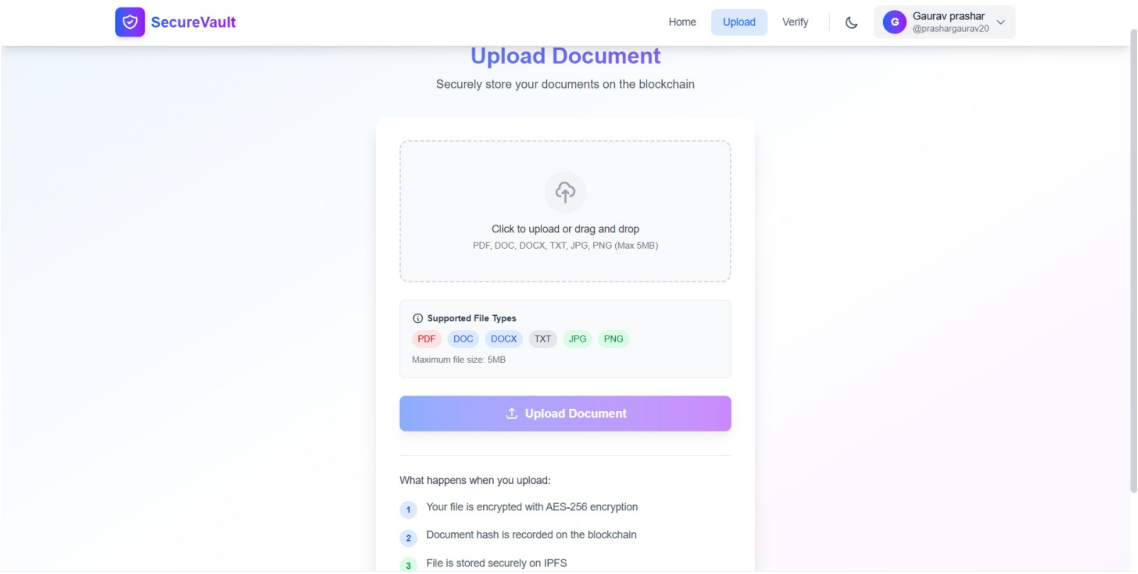


Figure 10: Document upload interface for secure file encryption and storage

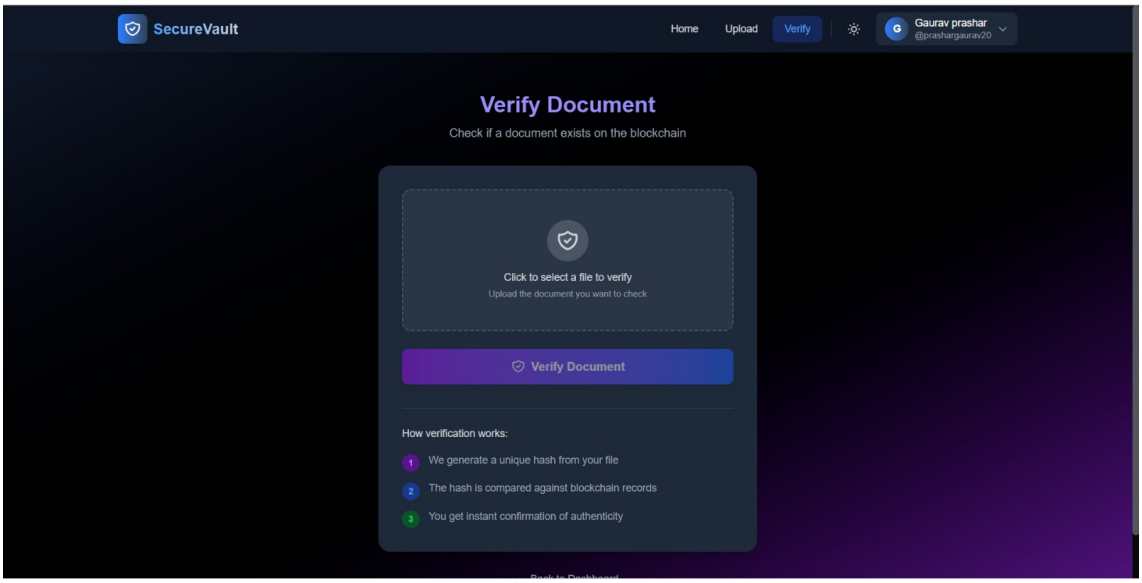


Figure 11: Document verification interface for validating file authenticity using blockchain records

References

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. London, UK: Pearson, 2017.

[2] National Institute of Standards and Technology, "Fips 197: Advanced encryption standard (aes)," 2001, federal Information Processing Standards Publication 197. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/197/final